Command Line

FDE Tutorial Sessions, Week 2

Disclaimer

These slides are supplementary materials provided by the tutor and are not endorsed by the teaching team. They are intended to summarize the key concepts, provide pointers to additional resources and make the tutorial sessions easier to follow for students who are unfamiliar with the topic.

Please note that, in the event of any differences, the official lecture slides, solution sheets and other materials uploaded to Moodle should take precedence, particularly when preparing for exams.

Shell

Shell = program that provides a CLI (=*command line interface*) for a unix-like OS, and a command language to be used interactively/as a script

Src: https://www.linfo.org/shell.html

`man` (Manual)

Provides manual pages for the commands

Manual page of `man` (`man man`) →

```
MAN(1)                                Manual pager utils                                MAN(1)

NAME
       man - an interface to the system reference manuals

SYNOPSIS
       man [man options] [[section] page ...] ...
       man -k [apropos options] regexp ...
       man -K [man options] [section] term ...
       man -f [whatis options] page ...
       man -l [man options] file ...
       man -w|-W [man options] page ...

DESCRIPTION
       man  is  the system's manual pager.  Each page argument given to man is normally the name of a
       program, utility or function.  The manual page associated with each of these arguments is then
       found and displayed.  A section, if provided, will direct man to look only in that section  of
       the manual.  The default action is to search in all of the available sections following a pre-
       defined  order  (see  DEFAULTS), and to show only the first page found, even if page exists in
       several sections.

       The table below shows the section numbers of the manual followed by the types  of  pages  they
       contain.

       1   Executable programs or shell commands
       2   System calls (functions provided by the kernel)
       3   Library calls (functions within program libraries)
       4   Special files (usually found in /dev)
       5   File formats and conventions, e.g. /etc/passwd
       6   Games
       7   Miscellaneous   (including   macro  packages  and  conventions),  e.g.  man(7),  groff(7),
           man-pages(7)
       8   System administration commands (usually only for root)
       9   Kernel routines [Non standard]
```

Commands from the lecture

Which ones do you remember?

Commands from the lecture

- **sort**
- **head**
- **tail**
- **uniq**
- **cut**
- **wc**
- **shuf**
- **sed**
- **join**
- **awk**

`comm` (compare)

Compares two files line by line

- Output has three columns: only in file1, only in file2, in both
- Both inputs must be **sorted**

Options:

`(none)`	show all three columns
`-1`	hide column 1 (lines only in file1)
`-2`	hide column 2 (lines only in file2)
`-3`	hide column 3 (lines in both)

```
comm [options] file1 file2
```

`comm file1 file2`
three-column comparison

`comm -12 file1 file2`
lines common to both files

`comm -23 file1 file2`
lines only in file1

`comm -13 file1 file2`
lines only in file2

Src: https://www.computerhope.com/unix/ucomm.htm

`grep` (**g**lobally search for the **r**egular **e**xpression and **p**rint)

Searches for lines matching a given pattern (regex)

Options:

`-i`	ignore case
`-v`	invert match (exclude pattern)
`-o`	print only matching part
`-r`	recursive search through directories
`-E`	use extended regex

```
grep [options] pattern [file...]
```

```
grep 'img[0-9]*\.jpg' *.html
```
regex match across all .html files

```
grep -i '.jpg' file
```
case-insensitive match

```
grep -v '^warning' logfile
```
exclude lines starting with “warning”

Attendance sheet

`cat` (concatenate)

Displays or concatenates file contents

Options:

(none)	print file contents to stdout
-v	show non-printable characters (binary data)
-n	number all lines
-b	number non-empty lines only

```
cat [options] [file...]
```

cat file
print file contents

cat file1 file2
concatenate multiple files

zcat, bzcat, xzcat
read compressed files

`less`

Displays text one page at a time (pager)

Options:

`(none)`	view file with scrolling
`-S`	truncate long lines instead of wrapping

```
less [options] [file]
```

`less file`
open file in pager

`command | less`
view command output interactively

sort

Sorts lines of text files

- Handles files larger than main memory

Options:

`(none)`	lexicographic sort
`-n`	numeric sort
`-k <n>`	sort starting from field n
`-t <char>`	specify field separator
`-u`	unique (remove duplicates)
`-r`	sort in reverse order

```
sort [options] [file]
```

```
cat file | sort
```
basic sort

```
cat file | sort -n
```
numeric sort

```
cat file | sort -k 2
```
sort starting from field 2

```
cat file | sort -t ',' -k 3
```
sort by comma-separated fields, starting from field 3

```
cat file | sort -u
```
sort and remove duplicates

Src: https://linux.die.net/man/1/sort (note usage of -k !)

`head` / `tail`

Display the beginning or end of a file

Options:

`-<N>`	show first or last N lines
`-n +<N>`	start from line N (tail)
`-n -<N>`	show all but last N lines (head)

```
head [options] [file]
tail [options] [file]
```

`head -20 file`
first 20 lines

`tail -n +3 file`
everything but first 2 lines

`head -n -2 file`
everything but last 2 lines

`uniq` (unique)

Filters **adjacent** duplicate lines in **sorted** input

Options:

`(none)`	remove duplicate lines
`-c`	prefix lines with counts
`-d`	print duplicates only
`-u`	print unique lines only

```
uniq [options] [file]
```

```
sort file | uniq -c
```
count occurrences

```
sort file | uniq -u
```
show only lines that occur exactly once

`cut`

Extracts specific fields or character ranges

- Useful for selecting relevant columns in tabular data

Options:

`-f <list>`	select fields (default delimiter: tab)
`-d <char>`	use custom delimiter
`-c <list>`	select characters instead of fields

```
cut [options] [file]
```

`cut -f 1,3 file`
show fields 1 and 3

`cut -f 2-4 -d ',' file`
comma-delimited fields 2–4

`cut -c 1-10 file`
first 10 characters

`wc` (word count)

Counts lines, words, and bytes in files

Options:

`(none)`	show all stats
`-l`	count lines
`-w`	count words
`-c`	count bytes

```
wc [options] [file]
```

`wc file`
all stats

`wc -l file`
line count only

`shuf` (shuffle)

Randomly shuffles input lines

- Use case: sampling

Options:

`(none)`	shuffle all lines
`-n <N>`	output only N random lines

```
shuf [options] [file]
```

`shuf file`
randomize lines

`shuf file | head -10000`
sample 10,000 lines

`seq` (sequence)

Prints a sequence of numbers, one per line

- Useful for generating test input, e.g. `seq 1 100 | shuf -n 5`

Options:

`(none)`	count from 1 (or first) up to last
`-s <sep>`	separate numbers with sep instead of newline
`-w`	pad with leading zeros to equal width
`-f <fmt>`	use a printf-style format

`seq [options] [first [incr]] last`

`seq 5`
1 to 5

`seq 2 5`
2 to 5

`seq 1 2 10`
1, 3, 5, 7, 9 (step of 2)

`seq -s ',' 1 5`
1,2,3,4,5 on one line

`seq -w 8 10`
08, 09, 10

`sed` (stream editor)

Stream editor for text substitution and transformation

- Supports regex and multi-command edits
- Common for batch replacements

Options:

`s/old/new/`	replace first match per line
`s/old/new/g`	replace all matches
`s/old/new/I`	case-insensitive replace

```
sed [options] 's/pattern/replacement/[flags]' [file]
```

`sed 's/old/new/' file`
replace first occurrence

`sed 's/old/new/g' file`
replace all occurrences

`sed 's/IMG_\([0-9]*\)\.JPG/image\1\.jpg/g'`
backreferences

join

Combines **sorted** input files on a common field (like **SQL joins**)

Options:

-1 <n>	join field from file1
-2 <n>	join field from file2
-t <char>	field separator
-a <n>	include unpaired lines from file n
`-a 1`	left outer join
`-a 2`	right outer join
`-a 1 -a 2`	full outer join
-v <n>	only unpaired lines from file n
`-v 1`	left anti join
`-v 2`	right anti join

join [options] file1 file2

join -1 2 -2 1 file1 file2

join on columns 2 and 1

join -1 2 -2 1 -a 1 file1 file2

left outer join

Src: https://blog.robertelder.org/intro-to-join-command/

awk

Executes a program **for each line of input**

- Very powerful for data extraction, aggregation, and transformation
- Can replace combinations of grep, cut, and sed for complex logic

```
awk 'BEGIN {init} {action} END {final}' [file]
```

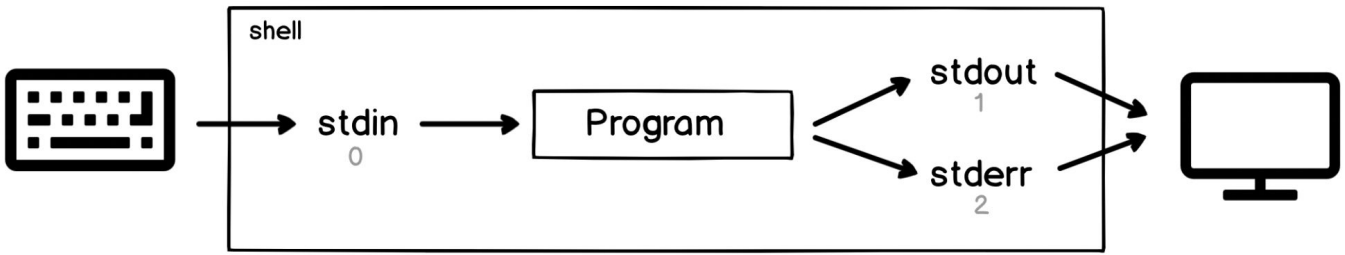
```
awk 'END {print NR}' file
```
count lines

```
awk '{sum += $2} END {print sum}' file
```
sum column 2

```
awk '{sum += $2; count++} END {print sum/count}' file
```
average column 2

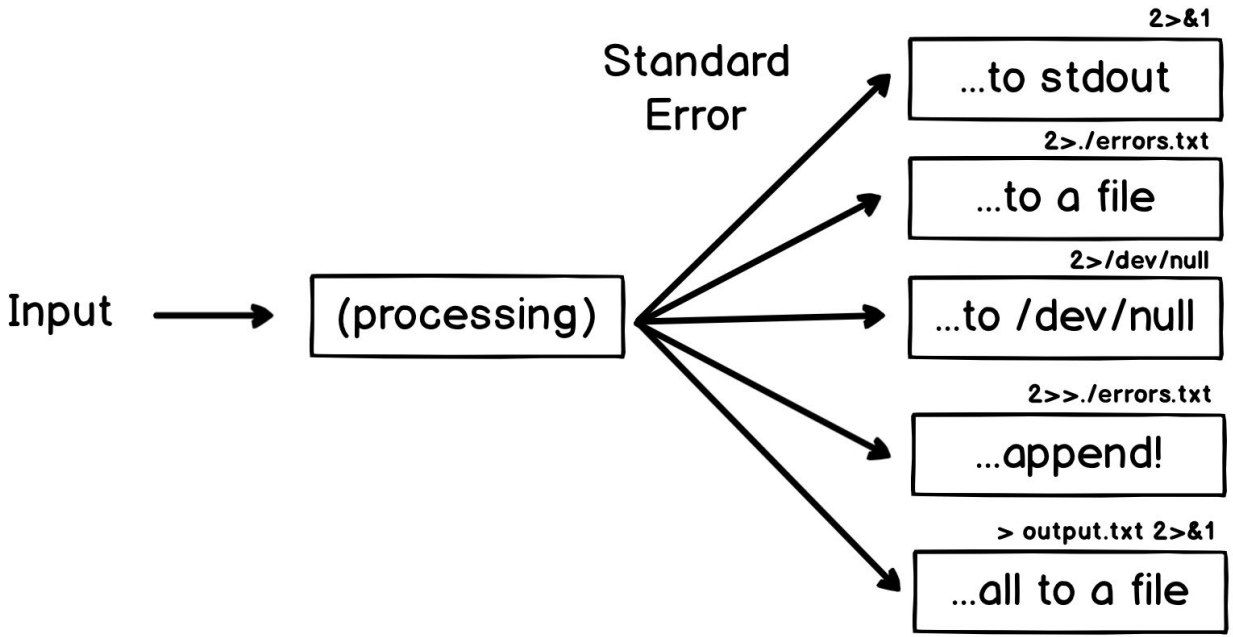
```
awk '{if ($1 > 10) sum += $2} END {print sum}' file
```
conditional sum

Pipe



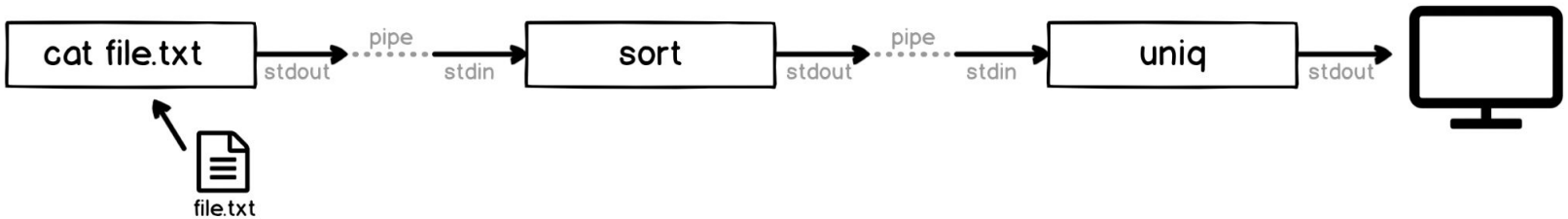
Img src: https://effective-shell.com/part-2-core-skills/thinking-in-pipelines/

Pipe



Img src: https://effective-shell.com/part-2-core-skills/thinking-in-pipelines/

Pipe



`command1 | command2`
pass stdout of *command1* to stdin of *command2*

Img src: https://effective-shell.com/part-2-core-skills/thinking-in-pipelines/

Redirection

Process substitution:

`command1 <(command2)`
use output of command2 as input to command1

Redirection:

`command1 < file`
input from file

`command1 > file`
write output to file (overwrite)

`command1 >> file`
append output to file

Control structures

`command1 && command2`

run command2 if command1 succeeds

`command1 || command2`

run command2 if command1 fails

Regular Expressions (Regex)

Regular expressions (extended)

`^`	start of line
`$`	end of line
`.`	any single character
`*`	zero or more repetitions
`+`	one or more repetitions
`?`	zero or one occurrence

`{M,N}`	between M and N repetitions
`{M,}`	at least M repetitions
`{,N}`	at most N repetitions
`{M}`	exactly M repetitions

`[^ ]`	negation (match any character not in **set**)
`( … )`	group / capture
`\.`	escape special character

*Repetition applies to the element before

Capturing groups

Used to group several characters into one entity

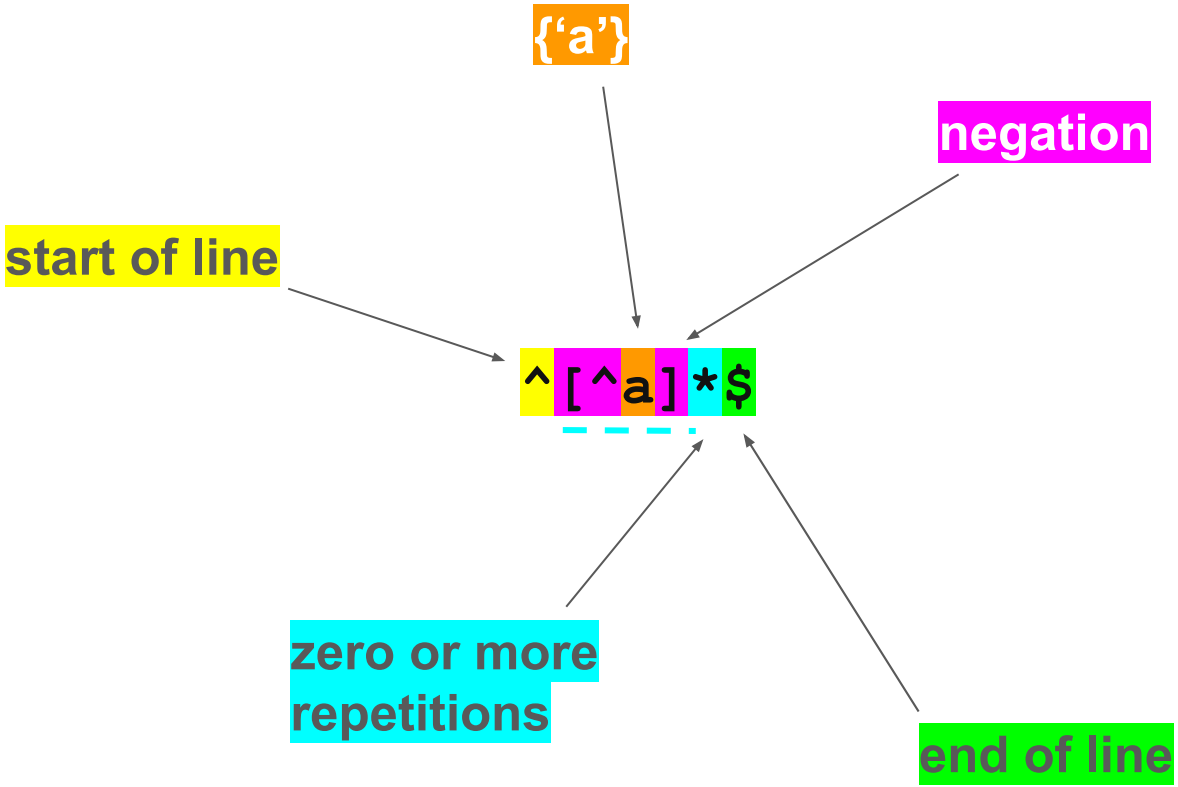
- Repetition applies to the whole group

  `ab+` → abbbbb… ; `(ab)+` → abababab…

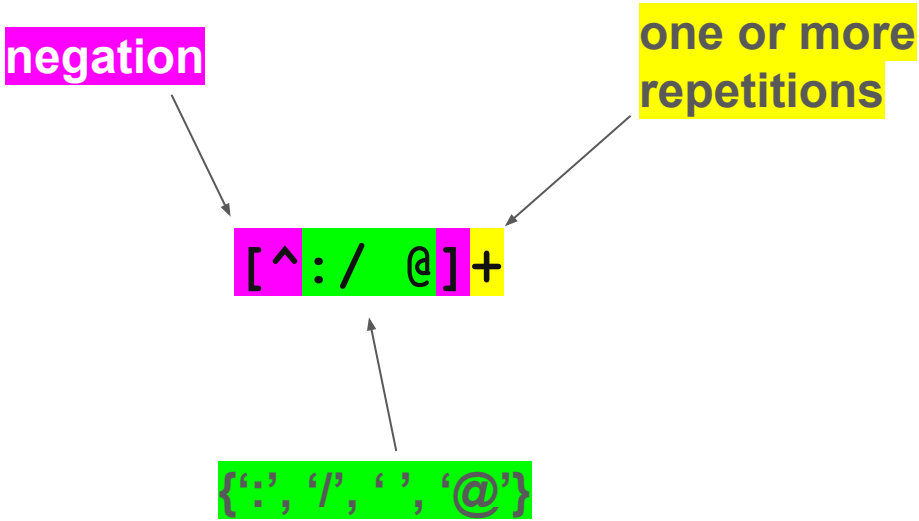
- Can use `|` inside of the group

  `(jpg|png)` → matches both “jpg” and “png”

- You can reuse what is captured with `\1`, `\2`…

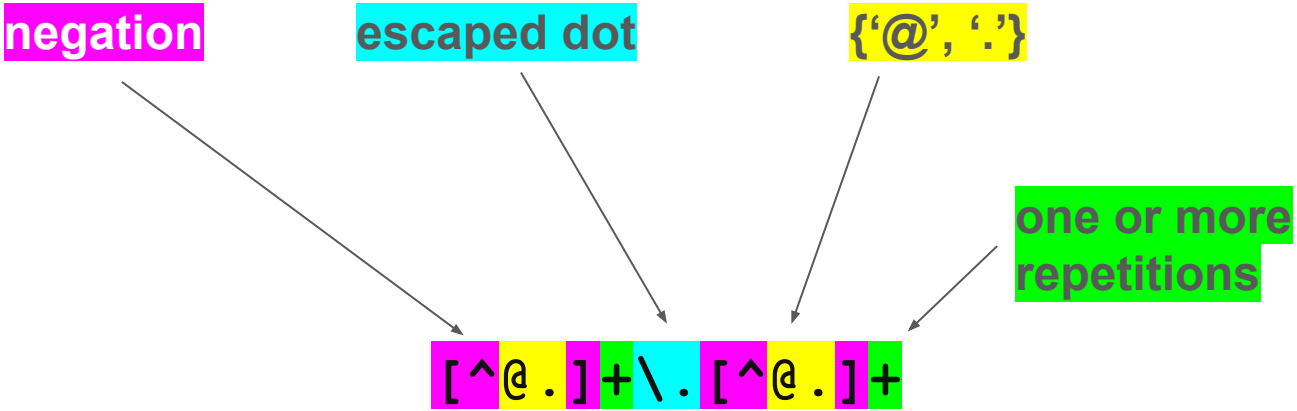


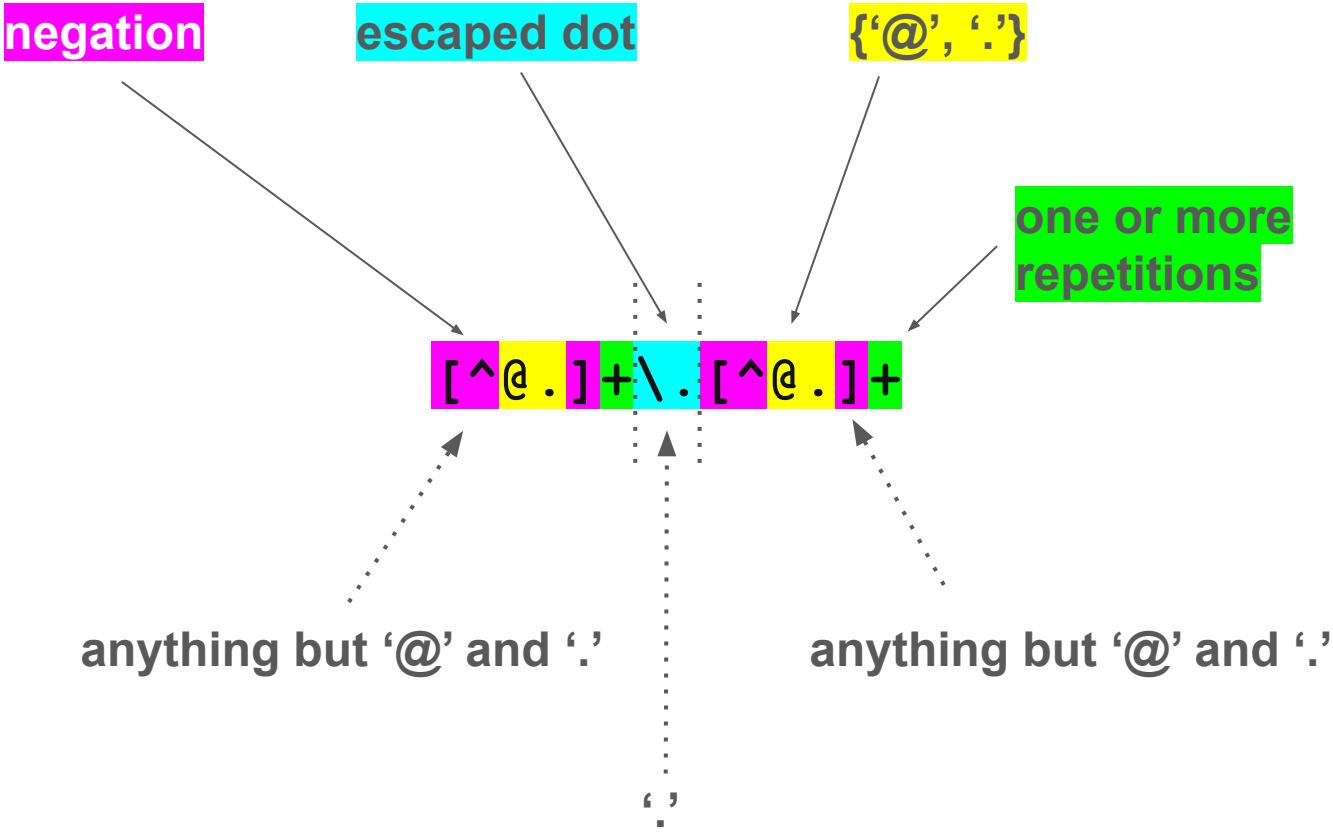
match any line without “a”

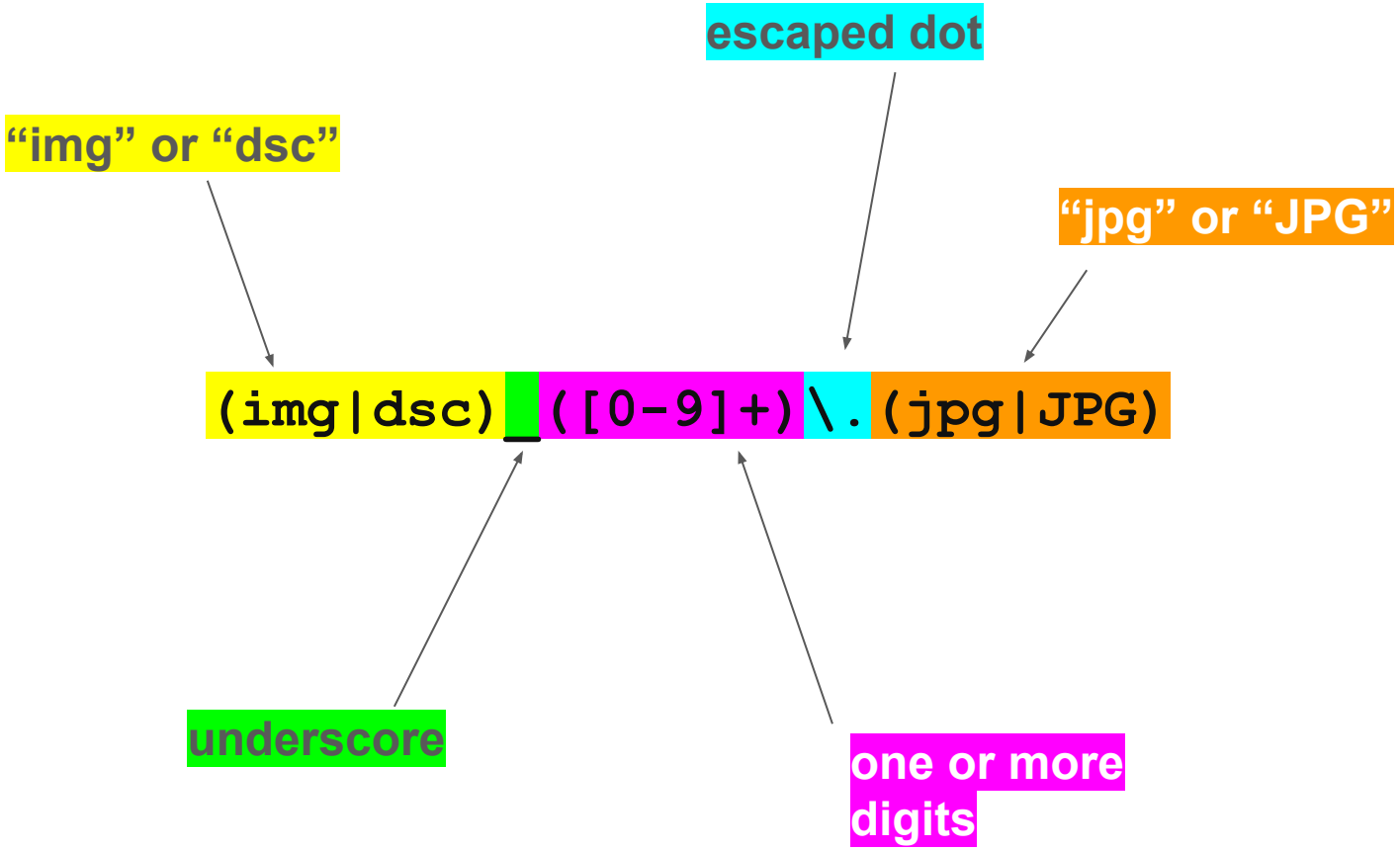


non-empty, no colons, slashes, spaces or @

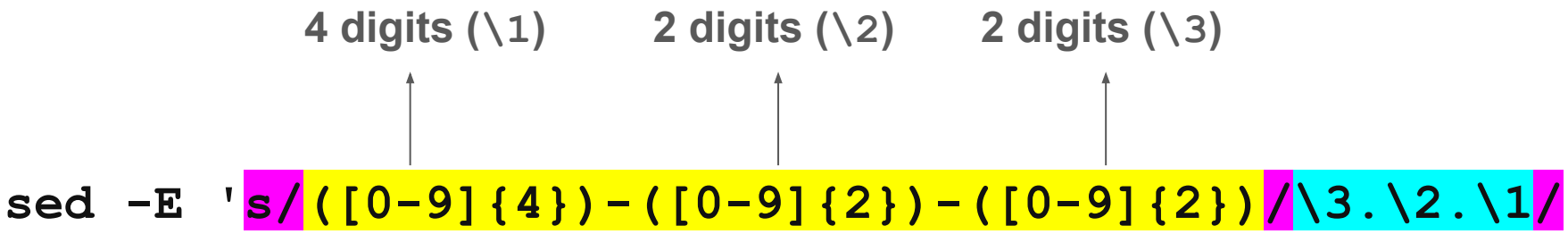
```
[^@.]+\.[^@.]+
```


negation
escaped dot
{‘@’, ‘.’}
one or more repetitions
[^@.]+\.[^@.]+


negation
escaped dot
{‘@’, ‘.’}
one or more repetitions
[^@.]+\.[^@.]+
anything but ‘@’ and ‘.’
‘.’
anything but ‘@’ and ‘.’


escaped dot
“img” or “dsc”
“jpg” or “JPG”
(img|dsc)_([0-9]+)\.(jpg|JPG)
underscore
one or more digits

```
sed -E 's/([0-9]{4})-([0-9]{2})-([0-9]{2})/\3.\2.\1/'
```



2026-10-30 → 30.10.2026

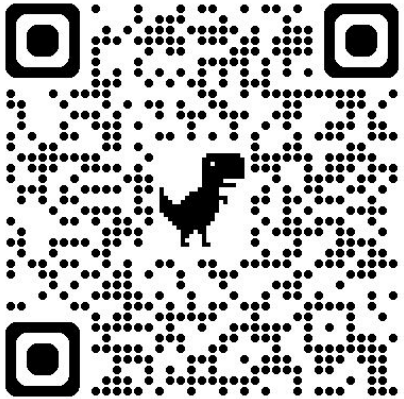
Useful links

Files and directories in Linux

For questions: veneta.kireva@tum.de

Thank you!

Link to these slides:



Next week: Combine C++ and command line
Make sure to have a Linux VM with C++ IDE; ***look at the sheet in advance***!